PARC2026におけるロボット基盤モデル（VLA）制御手法の包括的サーベイおよび勝ち筋に向けた技術戦略報告書

PARC2026の評価構造と技術的要件の包括的分析

Physical AI Robot Challenge 2026（PARC2026）は、Vision-Language-Action（VLA）基盤モデルを用いたロボットのエンドツーエンド制御における汎化性能を評価する競技である[1]。本コンペティションで持続的に上位を維持するためには、競技特有の多角的なスコア評価構造、単一GPUによる推論リソース制約、および段階的に増加するタスクの抽象度とドメインシフトに対する深い解釈が不可欠となる[1]。

コンペティションの評価関数は、単にタスクの成功率（Success Rate: SR）のみを測定するものではなく、実行効率、運動の滑らかさ、および安全性を包括した複合的な指標で構成されている[1]。具象的には、成功判定および衝突判定（Collision Penalty）がゲート乗数として機能する構造となっており、数式で表現すると以下の関係性に帰着される[1]。

$$\text{Total Score} = \text{Success Rate} \times (1 - \text{Collision Rate}) \times \sum_{i=1}^{5} w_i \cdot \text{Normalized Metric}_i$$

ここで選択される5つの正規化指標は、実行ステップ数、エンドエフェクタ（EEF）の軌道長、EEFの回転総量、Cartesian/Joint Jerk（急激な加速度変化）、およびSPARC（Spectral Arc Length: 軌道の滑らかさを示す周波数領域指標）である[1]。この数理構造が示唆する第2次インサイトは、タスク成功率が飽和する上位陣の戦いにおいては、JerkやSPARCに代表される軌道の滑らかさおよび動的効率性が最終的な順位の決定要因になるという点である[1]。いかに高精度にタスクを達成できたとしても、高周波な振動やカクつきを伴う動作を出力するモデルは著しく加算ポイントを失う設計となっている[1]。

本コンペティションの評価環境およびシステム制約は、モデル設計に対して厳格なバウンダリ条件を課している[1]。以下にその主要な仕様と技術的インプリケーションを整理する[1]。

評価・システム項目	仕様・制約条件	制御モデル設計への影響とインプリケーション
評価ハードウェア環境	単一 NVIDIA L4 GPU（VRAM 24GB）[1]	24GB VRAMのメモリ上限により、7Bクラス以上の大型パラメタモデルをフル精度で保持・推論することは困難であり、量化（Quantization）や効

		率的なLoRAアーキテクチャの導入が必須となる[1]。
推論応答遅延上限	1リクエスト（1ステップ）あたり10秒以内、1Track全評価あたり1時間以内[1]	自動回帰（Autoregressive）生成のステップ数を最小化する必要があり、並列デコーディングやアクション・チャンキング（Action Chunking）による高スループット化が要求される[1]。
提出パッケージ仕様	容量20GB以下のsubmission.zip（policy_server.py, model_weights/, requirements.txt等）[1]	評価実行中は外部ネットワークへのアクセスが遮断されるため、事前学習済み重みおよびすべての依存ライブラリを単一アーカイブ内に完結させる必要がある[1]。
段階的汎化トラック	Track 1（テクスチャ・照明摂動） Track 2（未知ドメイン・視点/位置ノイズ） Track 3（言語指示による既知タスクの新組合せ）[1]	単一過剰適合（Overfitting）モデルは無力化されるため、視覚的堅牢性、空間的不変性、および言語指示に対する組成表現力（Compositional Generalization）を段階的に獲得させなければならない[1]。
フェーズ別計算資源	本選1（60時間）/ 本選2（75時間）/ 本選3（120時間） （NVIDIA RTX PRO 6000 Blackwell環境提供）[2]	20日周期のスプリント評価と提供GPU時間を鑑み、ゼロからの事前学習ではなく、事前学習済み基盤モデルに対する効果的なSFT（教師あり微調整）および強化学習ポストトレーニングへ計算資源を集中させる必要がある[1]。

VLAおよびロボット制御における有望手法の包括的サーベイと比較分析

ロボットの視覚言語運動制御（Vision-Language-Action）領域における近年の研究動向は、大規模言語モデルの表現力を物理制御に転写する「自動回帰型VLA」から、連続的かつ平滑な動作系列を一括生成する「フローマッチング型・並列デコーディング型VLA」、さらには「強化学習（RL）ポストトレーニング」および「決定論的軌道後処理」の組み合わせへと急速にシフトしている[4]。本競技の基盤

ベンチマークであるLIBEROおよびその拡張であるLIBERO-Plusにおいて顕著な実績を上げている候補手法群を体系的に比較・整理する[7]。
候補手法のカテゴリーは、モデル構造、動作出力ヘッドの形式、学習パラダイム、および後処理技術の観点から以下のように分類される[4]。

手法・モデル系列	アーキテクチャおよび動作メカニズム	**LIBERO / LIBERO-Plus** での達成成功率	推論速度および**VRAM**消費（**L4**基準）	**PARC2026**評価指標（**Jerk/SPARC**等）への寄与	技術的長所と懸念点
OpenVLA (Base)	LLaVAベースの7Bモデル。離散トークン化された動作（7-DoF）をLLMと同様に自動回帰的に生成[6]。	LIBERO-4スイート平均76.5%[7]。摂動（LIBERO-Plus）下で成功率が激減[12]。	推論速度が低速（約4.2 Hz）。自動回帰のオーバーヘッドが大きくVRAMを大きく消費[5]。	動作空間の離散トークン化に起因する関節・直交座標系のカクつきが生じやすく、JerkやSPARCが悪化しやすい[4]。	長所: 視覚言語アライメントと事前学習表現が強固[6]。 懸念: 推論遅延が大きく、軌道の平滑性に難がある[7]。
OpenVLA-OFT / OFT+	OpenVLAに並列デコーディング、連続動作表現（L_1 回帰/Diffusion）、およびFiLM言語融合を導入[7]。	LIBERO-4スイート平均**97.1%**（SOTAレベル）[7]。	極めて高速（**100+ Hz**）。LoRA運用により24GB VRAM内で十分な余量をもって動作[5]。	連続値直接出力と動作チャンキングの平均化効果により、急激な加速度変化が抑制されJerkが大幅減衰[4]。	長所: 精度・スループット・軌道平滑性の全領域で極めて優れる[7]。 懸念: 動作ヘッドの付け替えと再学習が必要[7]。
π_0 / $\pi_{0.5}$ **(OpenPI)**	VLMバックボーンにFlow-Matching Action Headを結合し、動作	LIBERO-4スイート平均94.2%（π_0）。複雑な接触・リトライ動作で強み	中速（10〜25 Flowステップ設定時に約10〜20 Hz）。L4上で安定稼	フローマッチングによる生成過程が本質的に平滑な軌道を形成するた	長所: 運営提供ベースラインコードと完全互換。柔軟な再試行能力

	系列を連続的ノイズ除去過程で生成[10]。	を発揮[8]。	働[10]。	め、Jerk/SPARCのスコアが高い[10]。	を保有[2]。 懸念: 算術ステップ数と推論遅延の精緻なチューニングが必要[10]。
SmolVLA / AtomVLA	450M〜4Bの軽量VLMをベースとし、パラメータ効率を高めつつ高い汎化性能を達成[4]。	AtomVLA: 93.0%（SFT）→ **97.0%**（Post-Training後）[13]。	極めて軽量・高速。VRAM使用量が少なく、単一L4上で複数モデルのアンサンブルが可能[4]。	制御周波数が高いためステップ効率に優れる。平滑性は出力ヘッドの設計に依存[4]。	長所: 短時間のGPU学習で高速に反復実験を回すことが可能[4]。 懸念: 極度の未確認ドメイン（Track 3）でのゼロショット表現力に上限がある[1]。
Post-Training RL (π_{RL} / GRPO)	SFT済みのVLAに対し、シミュレータ環境でGRPOやPPOを用いたポストトレーニング強化学習を適用[13]。	LIBERO-Long等の難関タスクで成功率を57.6% → **97.6%** へ劇的向上[14]。	推論モデル構造自体はSFTと同一であるため、推論速度およびVRAM消費量への悪影響はゼロ[13]。	報酬関数にJerkおよびステップ数ペナルティを埋め込むことで、モデル自体に滑らかな挙動を学習可能[1]。	長所: 分布外ドメインでの堅牢性と実行効率が最大化される[13]。 懸念: ローカルシミュレータの並列実行訓練パイプラインの構築が必要[14]。
Trajectory Post-Processing (EMA	モデルが出力した動作チャンクや	適切なパラメータ設定下では、タス	計算オーバーヘッドほぼゼロ（	**Jerk**および**SPARC**を直接補正し、	長所: 追加学習不要。競技ルール

/ Savitzky-Golay)	命令ベクトルに対し、決定論的な時系列フィルタをリアルタイム適用[3]。	ク成功率を極めて高く維持[1]。	CPUでミリ秒単位の処理）[4]。	**Jerkを57%～68%**直接削減[3]。	上完全に許可された確実にスコアを向上させる手法[1]。 懸念: 過剰なフィルタリングは目標追従の応答遅延を招く[4]。

特にお勧めする主要手法（4選）の深層解説と選定理由

PARC2026の競技特性（20日ごとのスプリント評価、単一L4 GPU環境、成功判定と軌道平滑性の積による複合スコア構造）を徹底的に考慮し、上位入賞に向けて最大のROI（投資対効果）をもたらす4つの核となる手法を厳選して詳解する[1]。

推奨手法1: OpenVLA-OFT / OFT+ Recipe (Optimized Fine-Tuning)

メカニズムと技術的特性

OpenVLA-OFTは、従来の言語モデル派生VLAが直面していた「自動回帰ステップに伴う遅延」および「離散トークン化に起因する動作の不連続性」を排除するために開発された最適化ファインチューニングの体系である[7]。本手法の核心的メカニズムは以下の3要素で構成される[7]。

第1に、並列デコーディングとアクション・チャンキング（Parallel Decoding & Action Chunking）の導入である[7]。トランスフォーマーの隠れ状態から、将来の複数タイムステップ（例: $H = 8 \sim 25$ ）のアクション系列を1回の順伝播で一括生成する[7]。

第2に、連続アクション表現と L_1 回帰損失の採用である[7]。動作空間を離散トークンとして量子化せず、連続値ベクトルとして直接モデルに回帰させることで、情報損失を回避する[7]。

第3に、FiLM (Feature-wise Linear Modulation) を介したマルチモーダル融合である[7]。言語命令のエンコーディングをビジョンバックボーンの中間特徴量に対して要素ごとに線形変換（スケールおよびシフト）として注入することで、言語指示に対する厳密なアライメントを実現する[7]。

選定理由および**PARC2026**における優位性

本手法を推奨する最大の理由は、標準的なLIBEROベンチマークにおいて平均成功率 **97.1%** という圧巻の最先端（SOTA）成績を叩き出している実績にある[7]。加えて、従来のOpenVLAと比較して動作生成スループットが26倍（109.7 Hz）に急増し、推論遅延がわずか0.07秒程度に抑えられる点は、PARC2026の「1リクエスト10秒」「1Track合計1時間」という厳格なタイムアウト制約に対して絶対的な安心感を提供する[1]。連続値出力とチャンキング平均化処理は、急激な制御入力の変化を自然に抑制するため、スコア関数における Jerk および SPARC 指標を無条件で高次元へシフトさせる強力な特性を備えている[1]。

推奨手法2: $\pi_{0.5}$ Base Policy with LoRA & Multi-Task Data Augmentation

メカニズムと技術的特性

$\pi_{0.5}$ は Physical Intelligence によってオープンソース化された最新鋭のVLAアーキテクチャであり、動作出力部に Flow-Matching Action Head を組み込んでいる[10]。フローマッチングは拡散モデル（Diffusion Policy）概念の発展形であり、ベクトル場（Vector Field）を学習することで、連続的なノイズ除去過程を経て未来のアクション系列を同時生成する[10]。PARC2026の公式運営が提供する本選ベースラインコード（PARC2026_final）は、正にこの $\pi_{0.5}$ （OpenPIおよびLeRobot基盤）のLoRA学習環境として構築されている[2]。

選定理由および**PARC2026**における優位性

本手法の最大の強みは、コンペティション開始時点からの「開発立ち上げ速度」と「安定性」にある[2]。運営から提供されている examples/pi05_libero_finetune スクリプトおよび事前検証済みチェックポイントを活用することで、最初のアクション予測パイプラインを即座に稼働させることが可能である[2]。フローマッチング型ヘッドは、物体の把持や隙間への差し込みといったマルチモーダルな接触不確定性が存在するタスクにおいて、多峰性（Multimodal）のアクション分布を精緻に表現できる特性を持つ[10]。このため、動作失敗時に自律的にアプローチをやり直す「リトライ行動（Retrying Behavior）」が発現しやすい[15]。割り当てられたBlackwell GPU資源の範囲内で、LoRAランク（$r = 32 \sim 64$ ）の調整とデータ拡張を適用することで、短期間で確実にスコア（Track 1ベースライン0.286から0.80以上）へ到達可能な現実解となる[1]。

推奨手法3: シミュレータ活用によるポストトレーニング強化学習（**GRPO /** π_{RL} ）

メカニズムと技術的特性

教師あり学習（SFT）のみで訓練されたVLAモデルは、学習データに存在しない初期状態やわずかな視点ノイズに遭遇した際、予測誤差が時間とともに累積する累積誤差（Compounding Error）の問題を回避できない[11]。この課題を根本から解決するのが、SFT済みチェックポイントを出発点とし、ローカル環境のシミュレータ上でポリシーを強化学習（RL）によって最適化するポストトレーニング（Post-Training）パラダイムである[13]。

特に注目されるアルゴリズムが GRPO（Group Relative Policy Optimization）である[13]。GRPOでは、別途巨大な価値関数モデル（Critic）を保持することなく、同一の観測状態からポリシーが生成した複数のアクション系列グループ（Group）の相対的な報酬差分に基づいて直接方策勾配を更新する[13]。

選定理由および**PARC2026**における優位性

第一に、公式のQ&Aガイドラインにおいて「配布ローカル評価環境を使った強化学習」は禁止事項に含まれないことが明示されており、ルールの範囲内で最大限の競合優位性を構築できる[1]。

第二に、近年の研究（π_{RL} および AtomVLA）が実証している通り、SFT単体では成功率が50〜70%程度に伸び悩む高難度タスク（LIBERO-LongやLIBERO-Plusのカメラ視点・照明摂動環境）に対し、RLポストトレーニングを適用することで成功率を **97%〜98%** という極限値まで跳ね上げることが可能である[13]。

第三に、PARC2026独自の複合スコア関数をそのままRLの報酬関数（Reward Function）として定義できる点である[1]。タスク成功時にプラス報酬を与えるだけでなく、Jerkやステップ数、軌道長に対するペナルティを報酬設計に直接組み込むことで、モデル自身に「成功しつつ、最も滑らかで最短の動作を実行する」ように挙動を自己学習させることができる[1]。

推奨手法4: アクション・チャンキング結合型 Trajectory Post-Processing（EMA / Savitzky-Golay フィルタリング）

メカニズムと技術的特性

本手法は、モデル内部の改変ではなく、policy_server.py 内で推論された直交座標または関節角のアクション系列 $[dx, dy, dz, droll, dpitch, dyaw, gripper]$ に対して、リアルタイムで適用する決定論的信号処理アルゴリズムである[1]。

具体的には、過去の実行アクション a_{t-1}^{exec} と現在の予測アクション a_t^{pred} を指数平滑化係数 $\alpha \in [0.2, 0.4]$ を用いて補間する Exponential Moving Average（EMA）処理を適用する[4]。

$$a_t^{\text{exec}} = \alpha \cdot a_t^{\text{pred}} + (1 - \alpha) \cdot a_{t-1}^{\text{exec}}$$

さらに、モデルがアクション・チャンキングによって一括生成した未来の系列空間（例: $H = 10$ ）に対して、低位多項式適合を行う Savitzky-Golay フィルタを適用し、高周波振動を取り除いた状態でロボットのコントローラへ送信する[3]。

選定理由および**PARC2026**における優位性

公式ルールQ&Aにおいて「全タスク共通・決定論的なactionの後段補正」は禁止事項に該当しないことが明確に回答されているため、100%安全に導入できる[1]。実証実験および幾何解析データによれば、適切なEMAおよびSavitzky-Golayフィルタの導入によって、タスク成功率を一切損なうことなく、軌道の Jerk（加速度変化）を **57%〜68%** 削減 できることが証明されている[3]。本手法はモデルの再学習を一切必要とせず、推論コード内に数行の算術演算を挿入するだけで完了するため、すべての提出モデルに対する即効性のあるスコア底上げ策として機能する[1]。

フェーズ別（20日×3期間）の技術開発ロードマップ

PARC2026の本選構造は、約20日ごとに段階的な評価（本選1: 8/24-9/17, 本選2: 9/28-10/22, 本選3: 10/28-11/24）が行われ、生存者が絞り込まれる勝ち残り方式となっている[1]。与えられた段階的GPU時間（本選1: 60時間、本選2: 75時間、本選3: 120時間）を考慮し、フェーズごとに最大のアウトプットを創出するマイルストーンを以下のように提示する[1]。

フェーズ別の実行プロセス、主要成果物、ターゲット指標、およびGPUリソース配分戦略の全体像は下表の通りである[1]。

フェーズ（期間）	主な開発テーマと具体的実行プ	必須の納品物・提出物	ターゲット指標および目標スコ	**Blackwell GPU** 時間配分戦略

	ロセス		ア	
第**1**フェーズ （1〜20日目） 本選1 評価フェーズ	**Track 1**（同一ドメイン）の完全攻略と基盤パイプラインの固定 1. Docker環境での validate_submission.py によるローカル検証の完了[1]。 2. lerobot/libero_combined_20hz データを用いた $\pi_{0.5}$ の LoRAファインチューニング[2]。 3. policy_server.py への EMA / Savitzky-Golay 後処理フィルタの実装とJerk低減の検証[1]。	・動作確認済みの submission.zip [cite: 1, 2, 9] ・本選1 リーダーボードおよび最終評価フォームへのアップロード[2]	**Track 1 Total Score: 0.85+** （ベースライン 0.286 からの飛躍）[2] Cartesian Jerk 50%以上削減[4]	合計 **60**時間 ・環境構築・検証: 5h ・ $\pi_{0.5}$ LoRA学習: 40h ・後処理・ハイパラ調整: 15h[2]
第**2**フェーズ （21〜40日目） 本選2 評価フェーズ	**Track 2**（未知ドメイン・摂動）攻略と強化学習の導入 1. Sylvest/LIBERO-plus データ（照明・テクスチャ・視点ノイズ）を用いた視覚堅	・本選2 各Track向け最適化モデル zip[2] ・ドメインシフト耐性評価ログ[2]	**Track 2 Total Score: 0.75+** （ベースライン 0.165 からの飛躍）[2] 未知視点での成功率 85%以上[12]	合計 **75**時間 ・データ拡張 SFT: 35h ・ OpenVLA-OFT 学習: 25h ・GRPO強化学習試行: 15h[2]

	牢学習[2]。 2. OpenVLA-OFTアーキテクチャへの移行、または $\pi_{0.5}$ の動作チャンク長最適化[2]。 3. ローカルシミュレータ環境上での GRPO ポストトレーニング強化学習の導入[1]。			
第3フェーズ （41〜60日目） 本選3 評価フェーズ	**Track 3**（言語新規組合せ）完全制覇とアンサンブル・レポート完遂 1. 言語指示のパラフレーズおよびFiLM層の再チューニングによる組成汎化最大化[1]。 2. 異構成モデル間の動作平均（Action Averaging）によるアンサンブルと量子化（BF16/INT8）[1]。 3. 単一L4 GPU環境での1時間連続ロードテス	・本選3 最終提出用 zip[2] ・PDF形式 2ページ以内の詳細技術レポート[1]	**Track 3 Total Score: 0.65+** （ベースライン 0.000 からの飛躍）[2] 全Track合計スコアでの上位入賞[1]	合計 **120**時間 ・Track 3 言語 SFT: 45h ・GRPO本格運用: 45h ・アンサンブル・最終検証: 30h[2]

	トおよび2ページ最終テクニカルレポートの作成[1]。			

第1フェーズ（本選1期間：1日目～20日目 / GPU上限 60時間）の詳細プロセス

第1フェーズのゴールは、Track 1（同一タスク・同一ドメインでのテクスチャ/照明摂動）において確実に高い成功率を収め、提出パイプライン上のシステムエラーリスクを完全に排除することにある[1]。
最初の3日間で、運営配布の環境（CUDA 13.0, PyTorch 2.11.0）と同等のDockerコンテナを立ち上げ、validate_submission.py を通過するダミーポリシーをOmnicampusのリーダーボードへ提出し、システムとの整合性を確認する[1]。

4日目から10日目にかけては、配布データセット lerobot/libero_combined_20hz を使用し、$\pi_{0.5}$ バックボーンに対してLoRAファインチューニングを実施する[2]。学習パラメータとして、ランク $r = 32$、アルファ $\alpha = 64$、学習率 2×10^{-4}、Gradient Accumulation Step $= 2 \sim 4$ を設定し、Step 5,000 および Step 10,000 のチェックポイントを抽出する[2]。
11日目から15日目にかけて、抽出したモデルの推論処理部（get_action メソッド）に決定論的後処理フィルタを挿入する[1]。具体的には、$\alpha = 0.3$ のEMAフィルタを組み込み、局所的な出力をスムーズ化する[4]。ローカル評価パイプライン（pipeline/cli.py）を実行し、フィルタ適用によって mean_cartesian_jerk が半減すること、および成功率が維持されることを定量的に確認する[1]。
16日目から20日目の期間で、本選1の最終評価フォームに対して、動作検証済みの submission.zip を余裕をもって完了基準でアップロードする[1]。

第2フェーズ（本選2期間：21日目～40日目 / GPU上限 75時間）の詳細プロセス

第2フェーズの目標は、Track 2（未知ドメイン、カメラ視点・初期位置・センサノイズの変更）に対する堅牢性を極限まで高めることである[1]。
21日目から25日目にかけて、配布データセットに含まれる Sylvest/LIBERO-plus（多様なドメイン摂動が加わったデータセット）を ~/data/ に展開し、SFTの学習パイプラインに組み込む[2]。視覚エンコーダの過剰適合（Catastrophic Forgetting）を防ぐため、ランダムカラージッターや微小なクロップといったデータ拡張処理を適用する[1]。

26日目から33日目にかけて、OpenVLA-OFT Recipe（並列連続デコーディング＋ L_1 回帰損失）の導入を推進する[7]。モデルのアクション・チャンキング長を $H = 8 \sim 16$ に拡張し、高速かつ平滑な動作出力を可能にする[7]。
34日目から38日目にかけて、SFTのみでは解ききれない高難度タスクに対し、ローカルシミュレータを用いた GRPO（Group Relative Policy Optimization）強化学習の実験を行う[1]。同一の観測状態から $G = 4$ 個の軌道を生成させ、複合スコア関数に合わせた報酬（成功判定、衝突判定、Jerkペナルティの組合せ）を与えることで、モデルの堅牢性を大幅に強化する[1]。

第3フェーズ（本選3期間：41日目～60日目 / GPU上限 120時間）の詳細プロセス

最終フェーズでは、Track 3（言語指示による新規タスクの組み合わせ汎化）の完遂と、総合順位の上位確定を目指す[1]。

41日目から48日目にかけて、Track 3の言語指示に対する組成汎化力を高めるため、命令文のパラフレーズ（類語置換・構文変更）を適用したデータでマルチタスク学習を実施する[1]。FiLM層のアライメントパラメータを重点的に更新し、未体験の言語指示に対しても正確なオブジェクトを認識してアプローチできるようにチューニングする[1]。

49日目から54日目にかけて、120時間の計算資源をフル活用し、異なる初期シードで学習したモデルの予測結果を平均化する軽量アンサンブル（Action Averaging）を構成する[1]。単一L4 GPU（24GB VRAM）のメモリ制約を逸脱しないよう、BF16精度での推論または量子化を適用し、スループットとメモリの双方を最適化する[1]。

55日目から60日目の最終期間では、単一L4 GPU環境のDocker内で1時間以上の評価ループを完走させ、メモリーリークやタイムアウトが起理し得ないことを最終確認する[1]。同時に、本選3の成績優秀者選考条件である2ページ以内のPDF技術レポート（学習データ、ハイパーパラメータ、モデル構成、ハッシュ値、ライセンス情報を網羅）を作成し、提出を完結させる[1]。

結論および期待される競合優位性

PARC2026において他者を引き離し勝利を掴むための本質は、単一の標準モデルをそのまま当てはめることではなく、評価関数の数理特性（成功率と軌道平滑性の積構造）とコンペティションの制約条件（L4 GPU、推論時間制限、ドメインシフト）に合致した多層的な技術スタックを構築することにある[1]。

第一に、最先端の成功率（97.1%）と超高速スループット（100+ Hz）を両立する **OpenVLA-OFT Recipe** または $\pi_{0.5}$ **(Flow-Matching)** を主軸アーキテクチャとして採用し、推論遅延と離散化カクつきの双方を根本から排除する[2]。

第二に、**EMA / Savitzky-Golay** 決定論的軌道平滑化 を policy_server.py 内に確実に挿入し、追加学習コストをかけずに Jerk を57%～68%直接削減することで、スコア関数の平滑化項目を極限まで引き上げる[1]。

第三に、**LIBERO-plus** データ拡張 とローカル環境での **GRPO** 強化学習ポストトレーニング を連動させ、SFTの限界を超えるドメインシフト耐性と高難度タスク成功率を獲得する[1]。

この3本の矢からなる明確な技術戦略を20日ごとの開発サイクルに当てはめて規律正しく実行することが、PARC2026における最良かつ再現性の高い勝利への最短ルートとなる[1]。

Works cited

1. PARC2026開発コンペ゜ティション_予選.pdf
2. PARC2026開発コンペ゜ティション_本選_v1.1.pdf
3. (PDF) Measurement properties of movement smoothness metrics for, https://www.researchgate.net/publication/380975225_Measurement_properties_of_movement_smoothness_metrics_for_upper_limb_reaching_movements_in_people_with_moderate_to_severe_subacute_stroke

4. Vision-Based Hand Shadowing for Robotic Manipulation via Inverse, https://arxiv.org/html/2603.11383v2
5. Deploy OpenVLA on GPU Cloud: Self-Host the Open Vision, https://www.spheron.network/blog/deploy-openvla-gpu-cloud/
6. OpenVLA: Open Source VLA for Robotics - Emergent Mind, https://www.emergentmind.com/topics/openvla
7. Fine-Tuning Vision-Language-Action Models: Optimizing Speed and, https://www.alphaxiv.org/abs/2502.19645
8. Fine-Tuning Vision-Language-Action Models: Optimizing Speed and, https://arxiv.org/html/2502.19645v1
9. LIBERO-plus - Hugging Face, https://huggingface.co/docs/lerobot/en/libero_plus
10. OpenVLA and OpenPI Compared: Open-Weight VLA Models, https://robocloud-dashboard.vercel.app/learn/blog/openvla-openpi-open-weight-vla
11. LIBERO-X: Robustness Litmus for Vision-Language-Action Models, https://arxiv.org/html/2602.06556v1
12. In-depth Robustness Analysis of Vision-Language-Action Models, https://arxiv.org/html/2510.13626v3
13. AtomVLA: Scalable Post-Training for Robotic Manipulation via, https://arxiv.org/html/2603.08519v1
14. Online RL Fine-tuning for Flow-based Vision-Language-Action Models, https://arxiv.org/html/2510.25889v2
15. Physical-Intelligence/openpi - GitHub, https://github.com/Physical-Intelligence/openpi
16. LIBERO-Object: Lifelong Learning in Robotic Manipulation, https://www.emergentmind.com/topics/libero-object
17. CVPR Poster LIBERO-Plus: A Progressive Robustness Benchmark, https://cvpr.thecvf.com/virtual/2026/poster/38735
18. ICCAS 2025 Program | Wednesday November 5, 2025, https://ras.papercept.net/conferences/conferences/ICCAS25/program/ICCAS25_ContentListWeb_2.html
19. Fine-Tuning Vision-Language-Action Models: Optimizing Speed and, https://openvla-oft.github.io/
20. LIBERO-Long — Robot Manipulation | CodeSOTA, https://www.codesota.com/browse/robots/robot-manipulation/libero-long
21. Zero-Shot Robot Learning from Minutes of Human Egocentric Videos, https://arxiv.org/html/2605.24934v1
22. Benchmarking Knowledge Transfer for Lifelong Robot Learning, https://www.researchgate.net/publication/401445601_LIBERO_Benchmarking_Knowledge_Transfer_for_Lifelong_Robot_Learning
23. Unified World Models: Coupling Video and Action Diffusion for, https://www.roboticsproceedings.org/rss21/p015.pdf
24. IRIS: Learning-Driven Task-Specific Cinema Robot Arm for ... - arXiv, https://arxiv.org/html/2602.17537v1
25. LIBERO-Plus: In-depth Robustness Analysis of Vision-Language,

https://sylvestf.github.io/LIBERO-plus/